Problem Statement: Write a CUDA program to perform two matrix additions.

Objectives

1. **Introduction to CUDA Programming**: Learn the basics of CUDA programming for parallel computing on NVIDIA GPUs.
2. **Matrix Addition Using CUDA**: Implement a CUDA program to perform matrix addition on GPU.
3. **Understand GPU Architecture and Memory Management**: Gain insights into CUDA threads, blocks, and memory management for efficient parallel execution.

Theory

CUDA (Compute Unified Device Architecture) is a parallel computing platform and application programming interface (API) model created by NVIDIA. It allows developers to use NVIDIA GPUs for general-purpose processing, significantly accelerating computational tasks compared to traditional CPU-based processing.

Key Concepts in CUDA Programming:

1. **Threads and Blocks**: CUDA organizes parallel execution using threads grouped into blocks. Each block executes independently, and threads within a block can cooperate using shared memory.
2. **Memory Hierarchy**: CUDA provides different types of memory:
    - **Global memory**: Accessed by all threads, but with higher latency.
    - **Shared memory**: Faster memory accessible by threads within the same block.
    - **Registers**: Private memory for each thread.
3. **Kernel Functions**: Functions executed on the GPU are called kernels. Kernels are written with a specific syntax (`__global__`) and are called from the host (CPU).

Conclusion

This CUDA program demonstrates how to perform matrix addition using GPU parallelism. By utilizing CUDA's kernel function and memory management, matrix addition can be efficiently parallelized across GPU threads. Understanding CUDA programming concepts such as threads, blocks, memory management, and kernel functions is essential for leveraging GPU acceleration in compute-intensive applications. This example serves as a foundational starting point for developing more complex CUDA programs for various computational tasks.

Source Code and Output /Screenshots:



